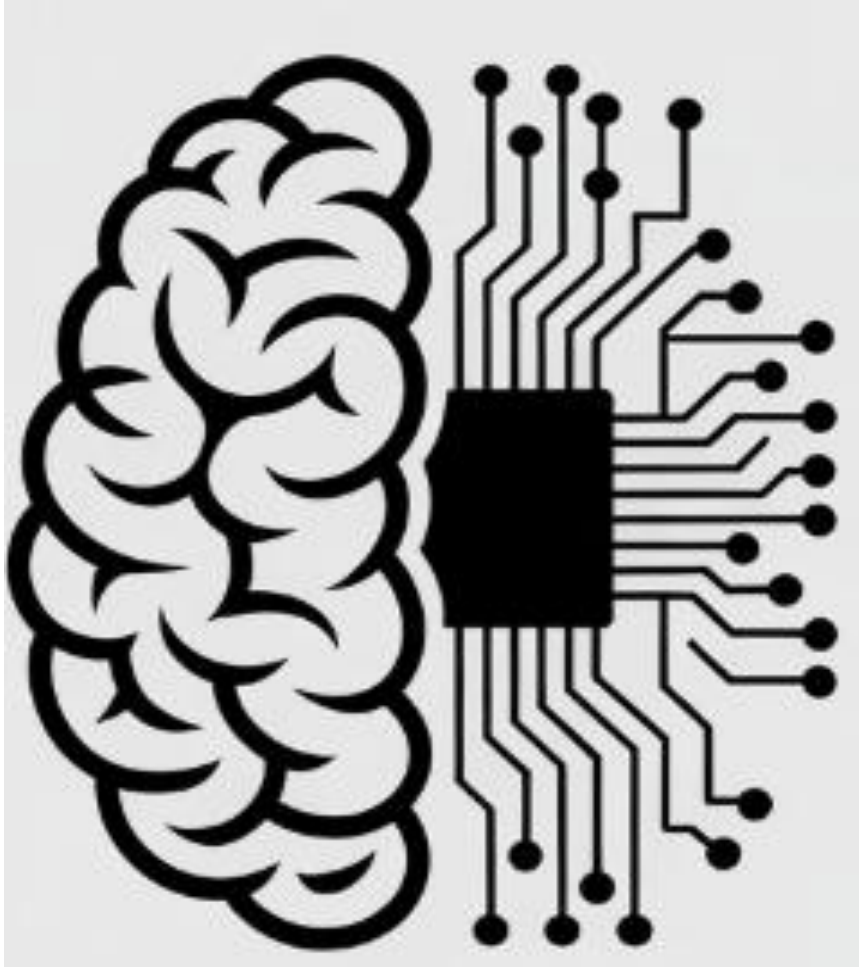
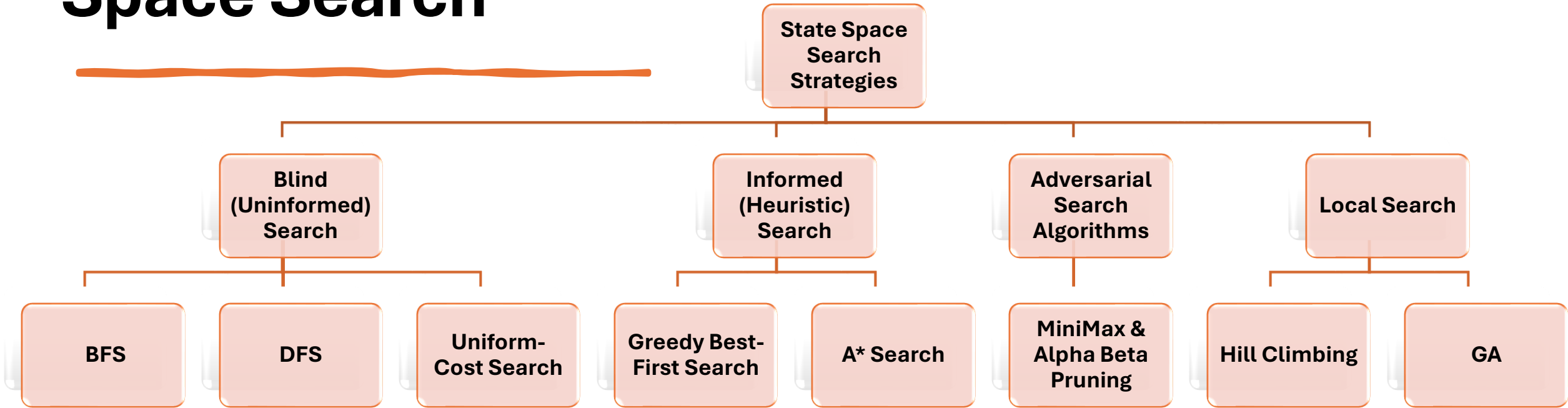




Hill Climbing

Dr. Sobia Tariq Javed

Strategies for State Space Search



Local Search

- **Previous search strategies** are designed to explore the search space in a structured and organized manner.
 - They maintain one or more paths in memory during the search process.
 - The algorithms record which states have been explored and which remain unexplored.
 - A solution is represented by a complete path from the initial state to the goal state.

Local Search

- In many **optimization problems**, the path to the goal is irrelevant; the goal state itself constitutes the solution.
- The **state space** consists of complete configurations rather than **partial paths**.
 - The objective is to find a configuration that satisfies given constraints (e.g., the **n-queens problem**).
 - In such problems, only the final configuration matters, not the order in which components are added or modified.
- Therefore, **local search algorithms** are suitable: they maintain a **single current state** and **iteratively** try to improve it.

Local Search

- Local search algorithms **iteratively improve a candidate solution** by making small, incremental changes and evaluating the objective function after each modification.
- The **objective function** guides the search by indicating whether a new solution is **better or worse** than the current one and by suggesting the direction of improvement.
- This process continues until a **satisfactory solution** is found or no further improvement is possible.

Local Search- Advantages

- Use very little memory
- Find reasonable solution in infinite(Continuous) State Space for which systematic algorithms are unsuitable

Local Search Algorithms are useful for Optimization Problems, in which aim is to find best state according to an **Objective Function**.

Hill Climbing

- Like climbing Everest in thick fog with amnesia

```
function HILL-CLIMBING(problem) returns a state that is a local maximum
  inputs: problem, a problem
  local variables: current, a node
                  neighbor, a node

  current ← MAKE-NODE(INITIAL-STATE[problem])
  loop do
    neighbor ← a highest-valued successor of current
    if VALUE[neighbor] ≤ VALUE[current] then return STATE[current]
    current ← neighbor
```

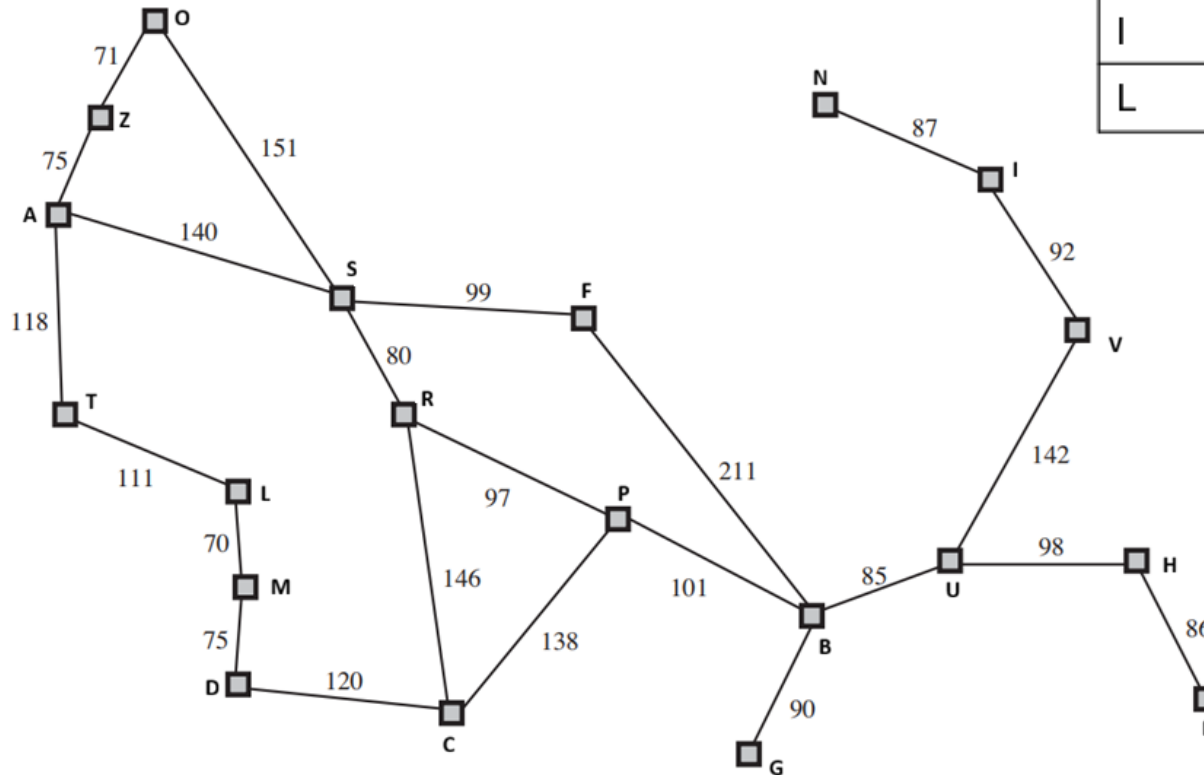
Hill Climbing search

- We can define our own objective function (minimize)

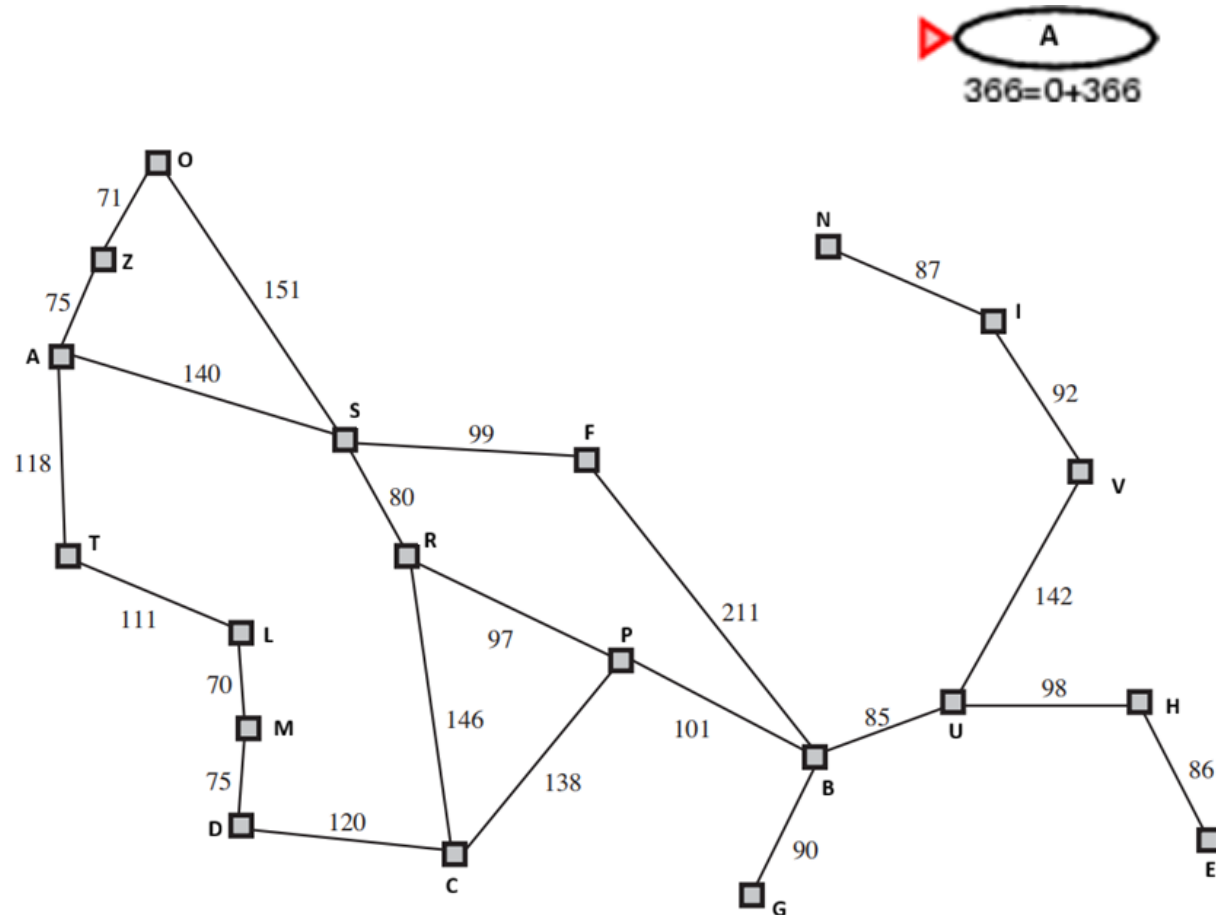
- $F(n) = g(n)$
- $F(n) = h(n)$
- $F(n) = g(n) + h(n)$

City	$h(n)$	City	$h(n)$
A	366	M	241
B	0	N	234
C	160	O	380
D	242	P	100
E	161	R	193
F	176	S	253
G	77	T	329
H	151	U	80
I	226	V	199
L	244	Z	374

$$F(n) = g(n) + h(n)$$

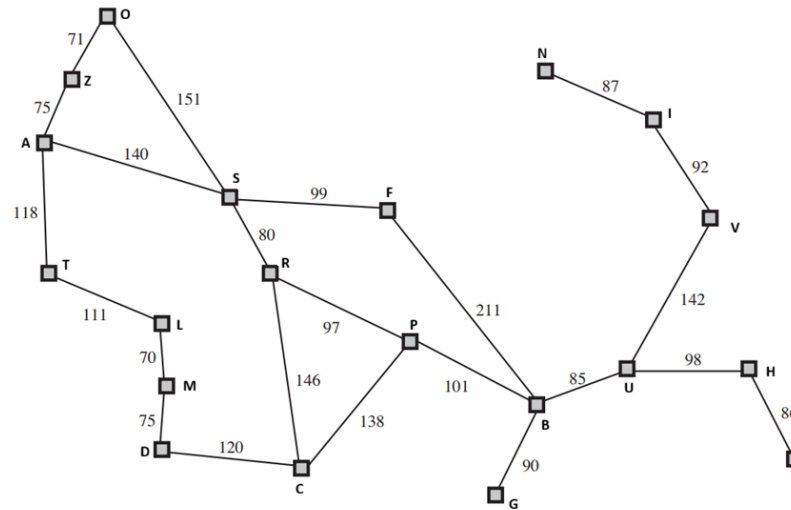


Hill Climbing search



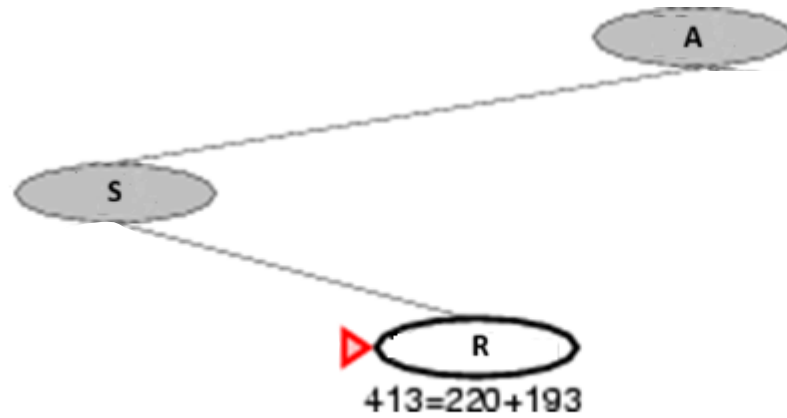
City	$h(n)$	City	$h(n)$
A	366	M	241
B	0	N	234
C	160	O	380
D	242	P	100
E	161	R	193
F	176	S	253
G	77	T	329
H	151	U	80
I	226	V	199
L	244	Z	374

Hill Climbing

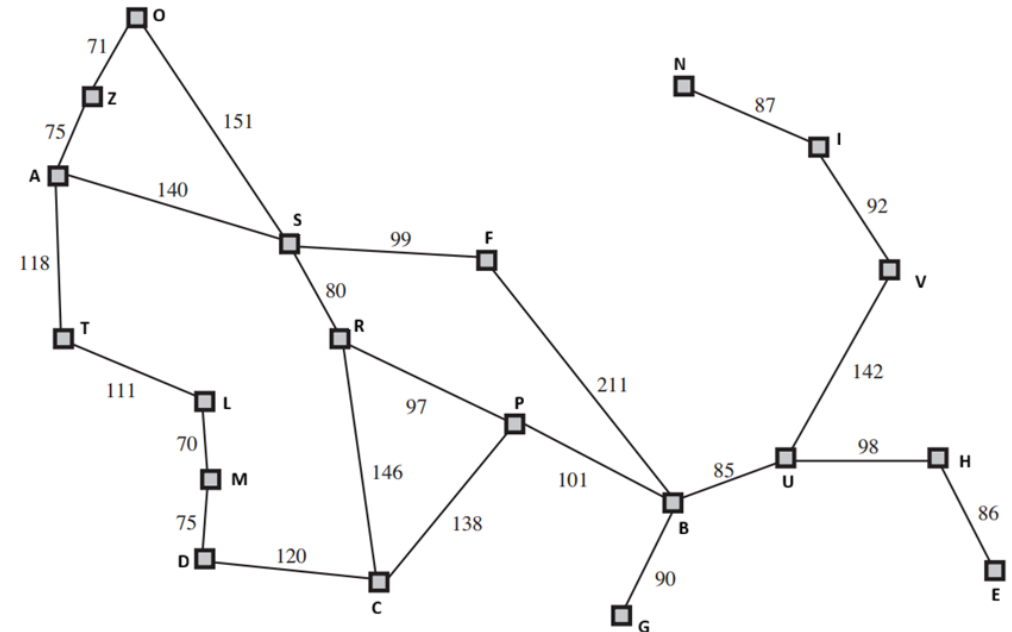


City	$h(n)$	City	$h(n)$
A	366	M	241
B	0	N	234
C	160	O	380
D	242	P	100
E	161	R	193
F	176	S	253
G	77	T	329
H	151	U	80
I	226	V	199
L	244	Z	374

Hill Climbing

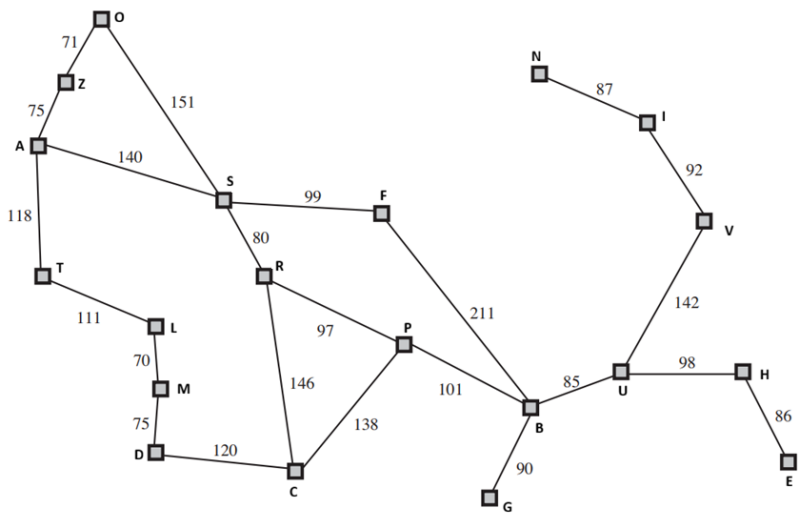
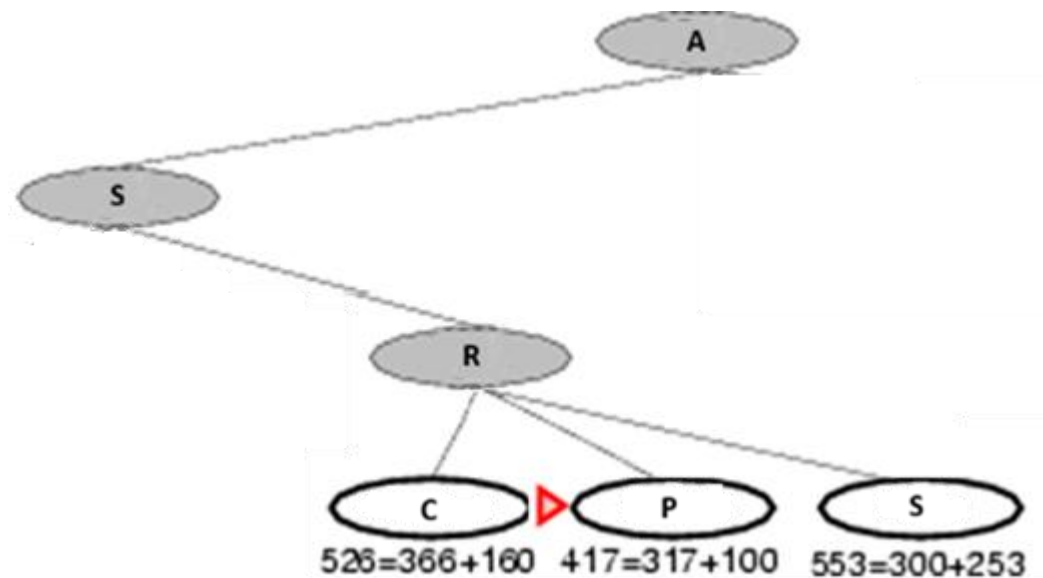


City	$h(n)$	City	$h(n)$
A	366	M	241
B	0	N	234
C	160	O	380
D	242	P	100
E	161	R	193
F	176	S	253
G	77	T	329
H	151	U	80
I	226	V	199
L	244	Z	374



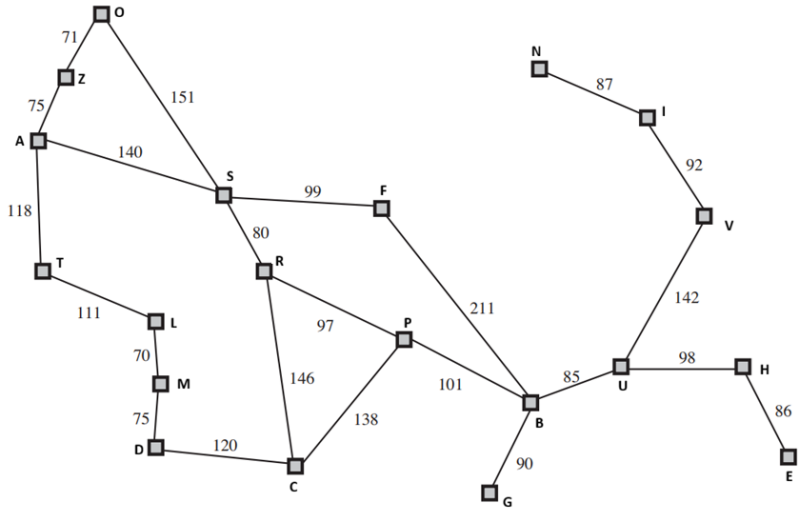
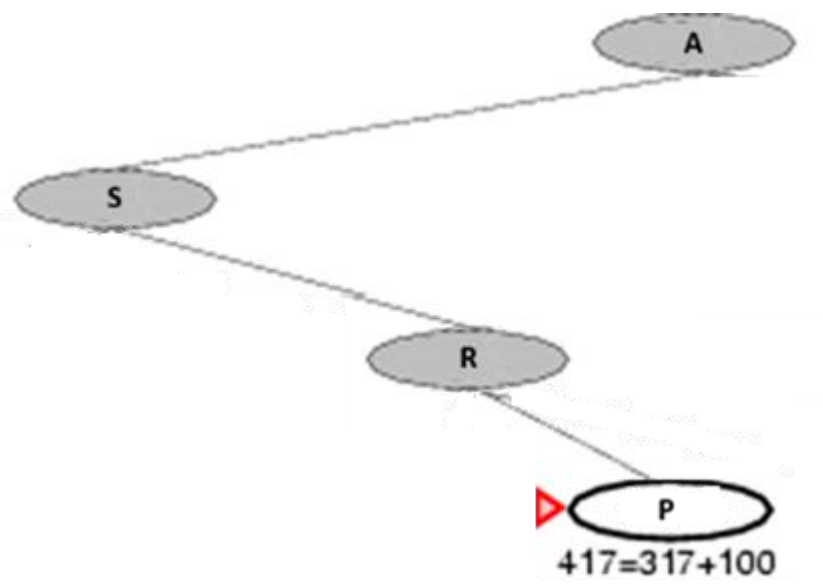
Hill Climbing

City	h(n)	City	h(n)
A	366	M	241
B	0	N	234
C	160	O	380
D	242	P	100
E	161	R	193
F	176	S	253
G	77	T	329
H	151	U	80
I	226	V	199
L	244	Z	374



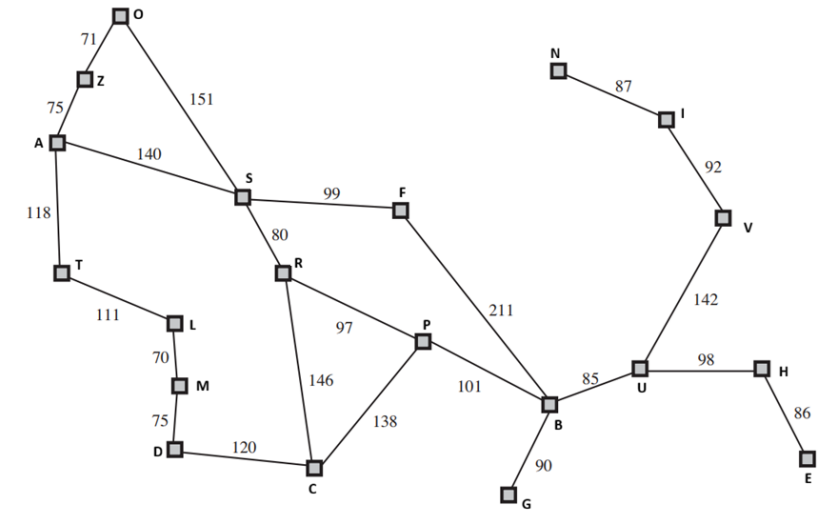
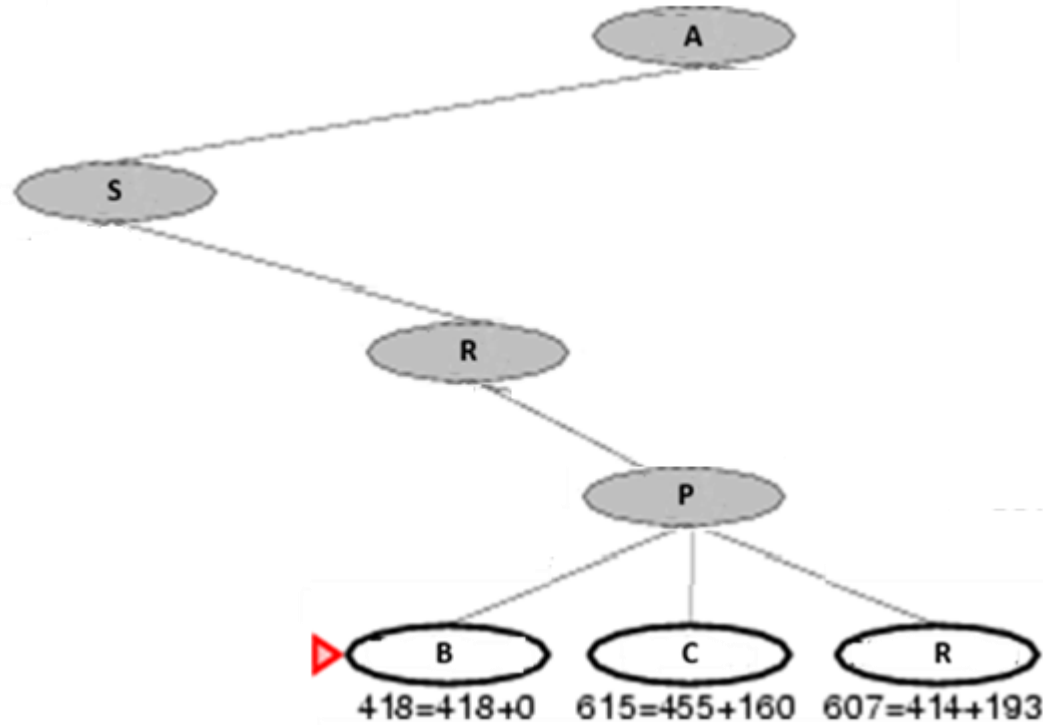
Hill Climbing

City	h(n)	City	h(n)
A	366	M	241
B	0	N	234
C	160	O	380
D	242	P	100
E	161	R	193
F	176	S	253
G	77	T	329
H	151	U	80
I	226	V	199
L	244	Z	374



Hill Climbing

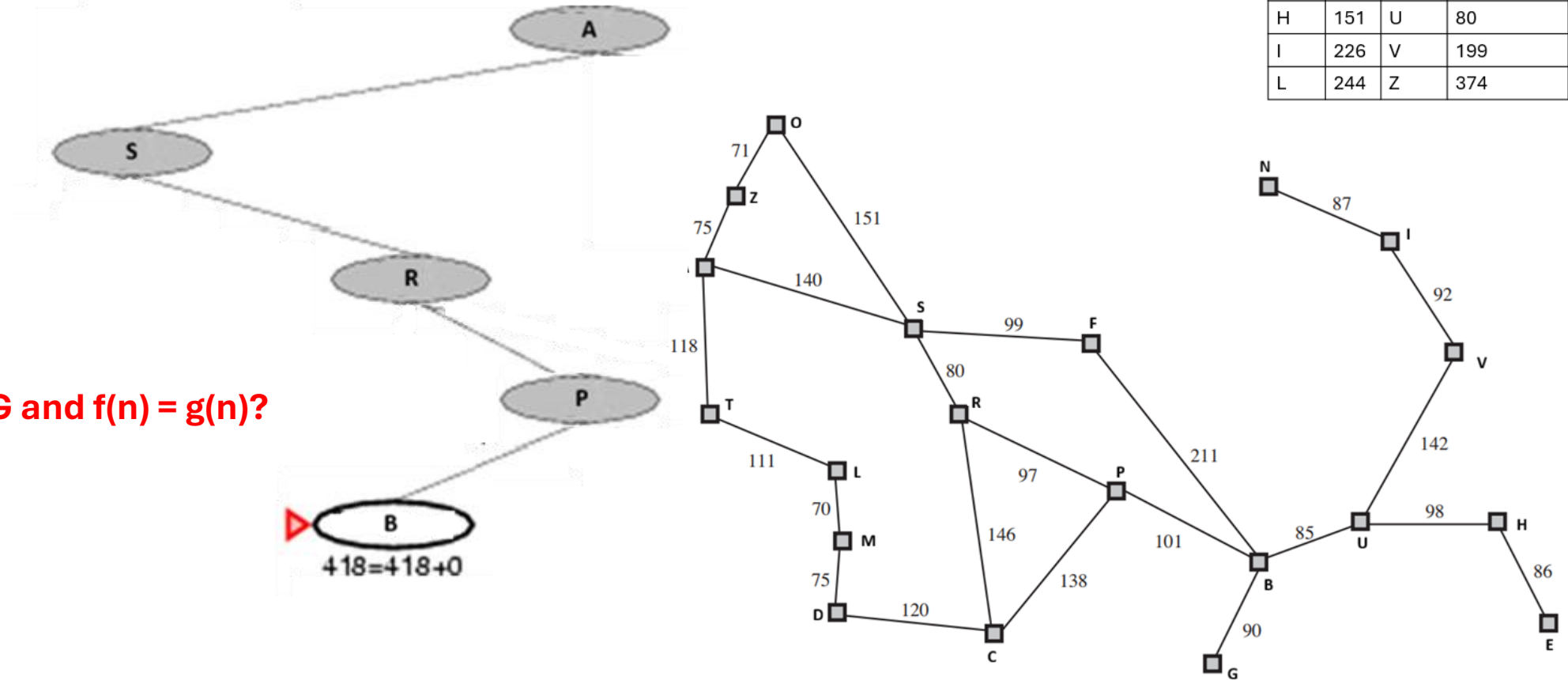
City	h(n)	City	h(n)
A	366	M	241
B	0	N	234
C	160	O	380
D	242	P	100
E	161	R	193
F	176	S	253
G	77	T	329
H	151	U	80
I	226	V	199
L	244	Z	374



Hill Climbing

City	h(n)	City	h(n)
A	366	M	241
B	0	N	234
C	160	O	380
D	242	P	100
E	161	R	193
F	176	S	253
G	77	T	329
H	151	U	80
I	226	V	199
L	244	Z	374

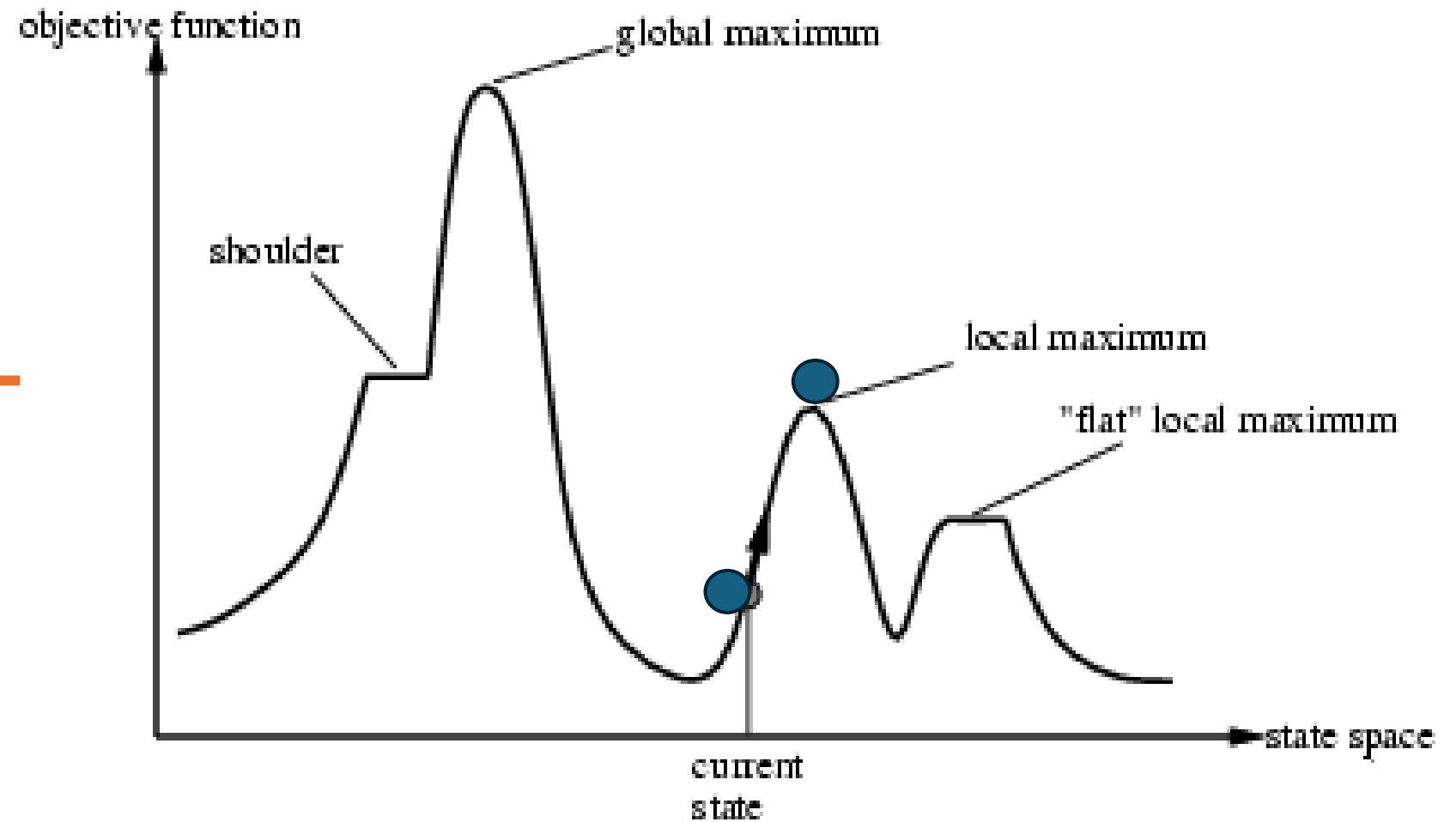
What if the Goal is G and $f(n) = g(n)$?



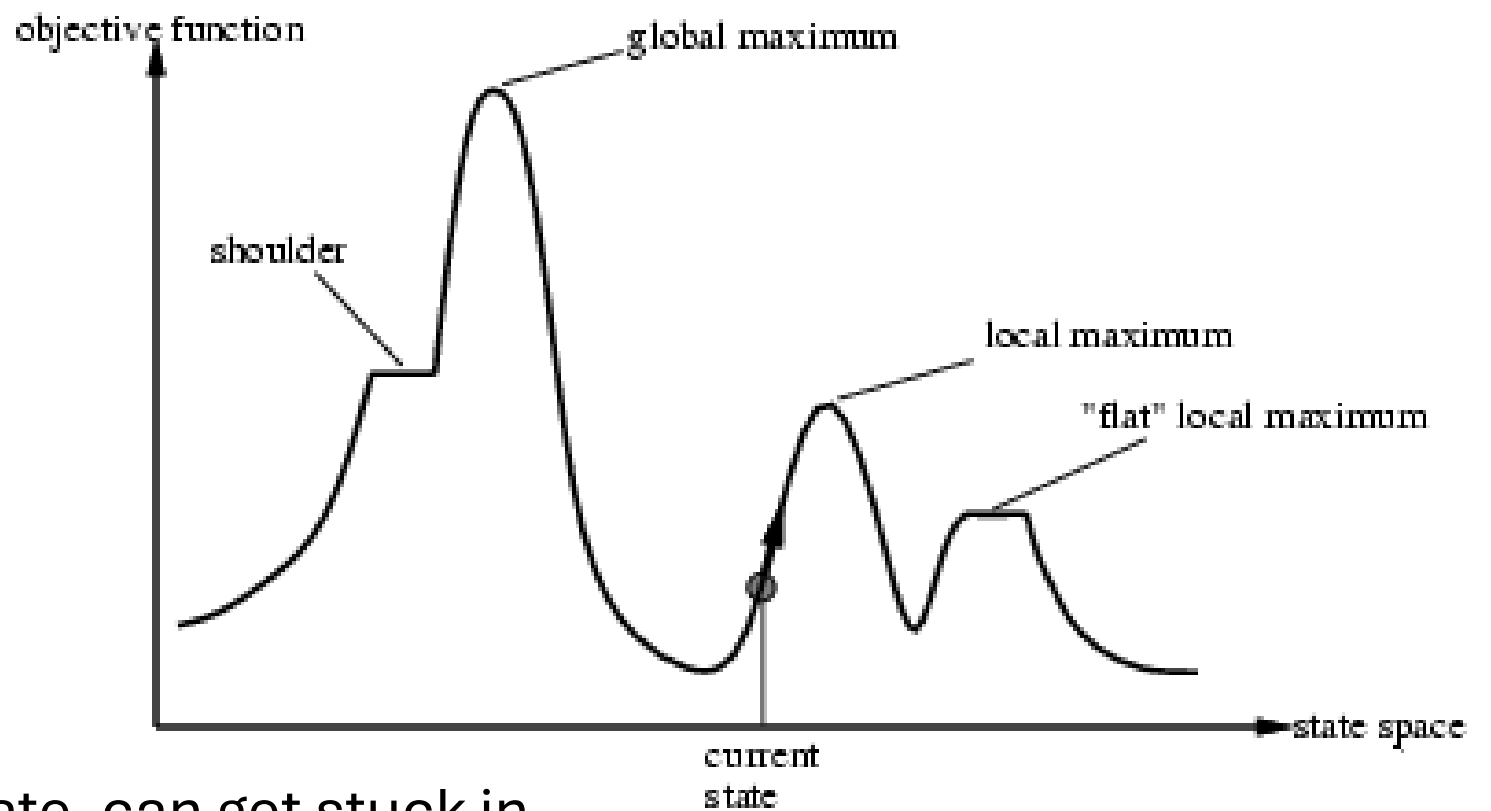
Hill Climbing

Problem: depending on initial state, can get stuck in

- **Local maxima**



Hill Climbing



Problem: depending on initial state, can get stuck in

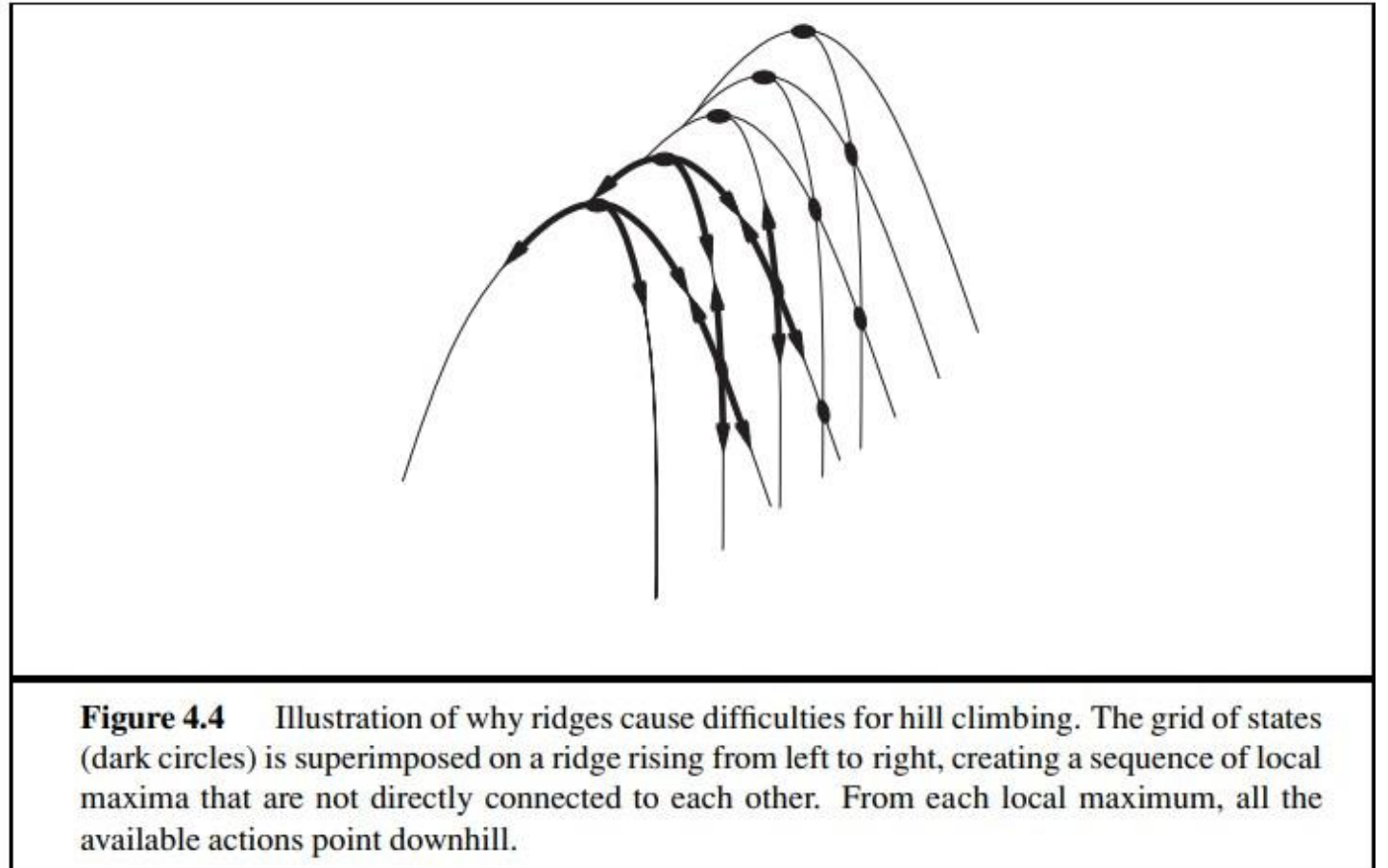
- **Local maxima**
- **Plateau**
 - It is a **flat region** of state space where neighboring states have the same value.
 - **Shoulder:** from which progress is possible

Hill Climbing

Problem: depending on initial state, can get stuck in

- **Local maxima**
- **Plateau**
- **Ridges**
 - Sequence of local maxima that is very difficult for greedy algorithms to navigate

Ridges result in a sequence of local maxima that is very difficult for greedy algorithms to navigate



Variants of Hill-climbing search

- **Simple Hill Climbing**

Examines neighboring states one by one and moves to the first neighbor that improves the current state.

- **Steepest-Ascent Hill Climbing**

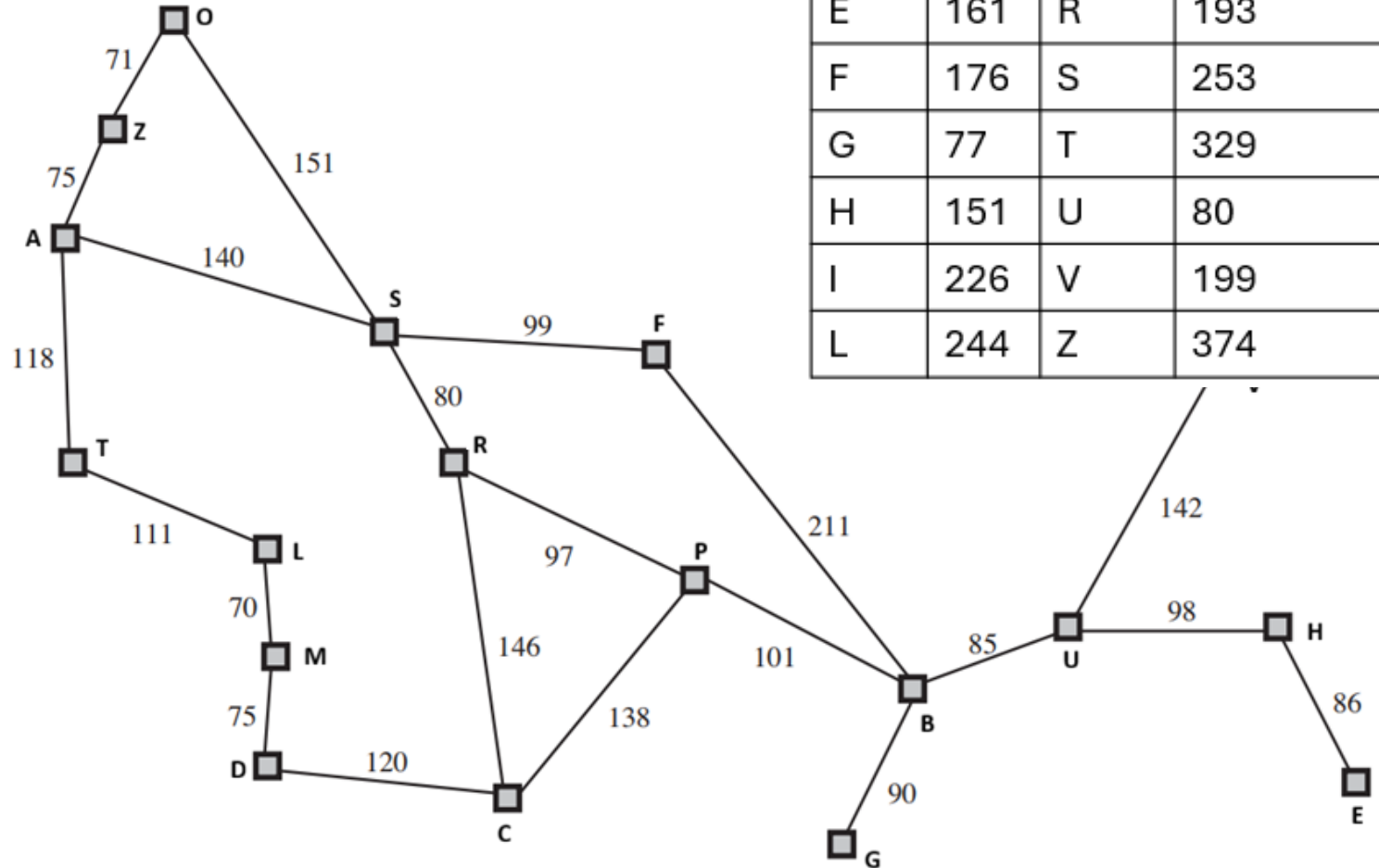
Evaluates all neighbors and moves to the one with the maximum improvement.

- **Stochastic Hill Climbing**

Randomly selects an improving neighbor, with higher probability for better moves.

Variants of Hill-climbing search

- Starting from S what will be the next state using
 - Simple Hill Climbing
 - Steepest-Ascent Hill Climbing
 - Stochastic Hill Climbing

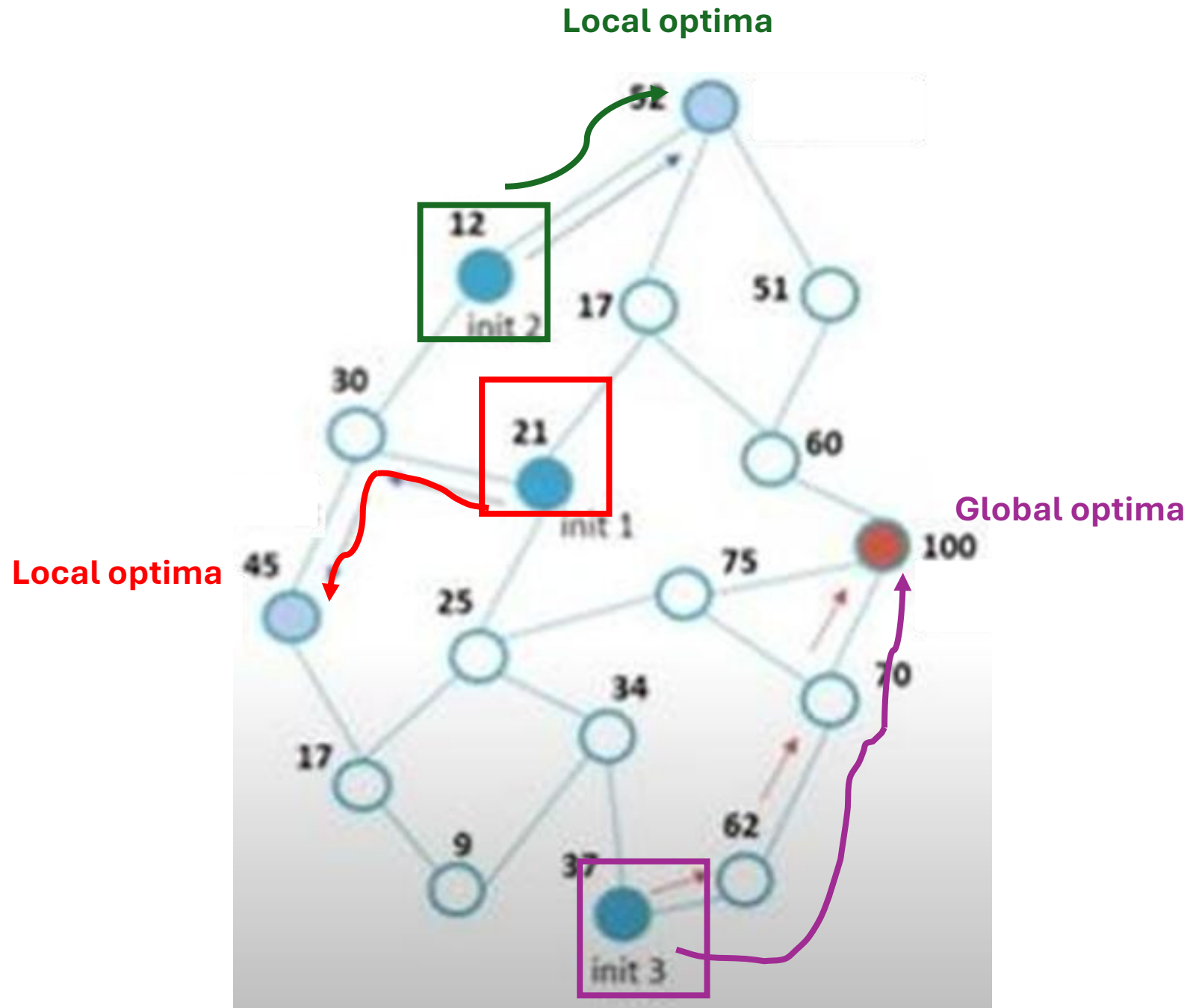


Variants of Hill-climbing search

- **Random-Restart Hill Climbing**

Runs hill climbing multiple times from different random initial states to escape local optima.

Random-Restart Hill Climbing



Hill Climbing

- **Complete? Optimal?**
- Hill climbing is sometimes called **greedy local search**
- Although greedy algorithms often perform well, hill climbing gets stuck when:
 - Local maxima/minima
 - Ridges
 - Plateau (shoulder or flat local maxima/minima)

Hill Climbing

- NP-hard problems typically have an exponential number of local maxima/minima to get stuck on
- A hill climbing algorithm that never makes “downhill” (or “uphill”) moves is guaranteed to be incomplete
- A purely random walk – moving to a successor chosen uniformly at random – is complete, but extremely inefficient

Hill Climbing-Exercise

- Suppose a corridor has 7 possible locations where we can place a WiFi router. The goal is to place the router where it serves the maximum number of users. Each location has a coverage score (number of users receiving strong signal). Each location whose number differs by 1 is considered a neighbor. For example, Location 2 has Location 1 and Location 3 as its neighbors, but Location 4 is not considered a neighbor.

- Using Hill climbing find the best location to place WiFi. Starting location = 2.

Location (x)	Users Covered $f(x)$
1	12
2	18
3	25
4	31
5	29
6	22
7	15

Hill Climbing-Exercise

- Using Hill climbing find the best location to place WiFi.
- Starting location = 2.
- The objective is :

Maximize $f(x)$

where $f(x)$ = number of users with strong signal.

Location (x)	Users Covered $f(x)$
1	12
2	18
3	25
4	31
5	29
6	22
7	15

Exercise –A* algorithm

- Execute the A* algorithms on the provided maze and build the search tree with the heuristic function provided below.
- In the maze, numbers represent food cells, and letters represent empty cells.
- Create the nodes from left to right in alphabetical order.

	D	1	2
A		E	F
B		3	
C	5	4	

Initial State

	D		
A		E	F
B			
C			

Goal State

$h(n)$ = Minimum Manhattan
distance from current position to
each dot and + no# of remaining
dots

References

- **CH 4- Beyond Classical Search (Section 4.1)**